Computational Physics Lectures

Week 9: Autodifferentiation in PyTorch

Bjørk Hammer
Department of Physics and Astronomy
Aarhus University
Denmark

Contents

9 PyTorch tutorial **3**

9.1 `tensor` . . . 3

9.2 `autograd.grad` . . . 3

9.3 `with torch.no_grad()` . . . 5

9.4 Higher derivatives . . . 5

9.5 `torch.nn.Module` I . . . 6

9.6 `torch.nn.Module` II . . . 8

9.7 `torch.nn.Module` III . . . 9

© Bjørk Hammer

October 8, 2026

Week 9

PyTorch tutorial

9.1 `tensor`

Working with `PyTorch` is to a large extent like working with `numpy`. You can write:

```
import torch
import matplotlib.pyplot as plt

xs = torch.tensor([1, 2, 3])
ys = xs**2
print(xs,ys)

fig, ax = plt.subplots()
ax.plot(xs,ys)
```

and get some arrays, now called **tensors**:

```
tensor([1, 2, 3]) tensor([1, 4, 9])
```

There are many helper functions for making tensors in PyTorch, e.g.:

```
torch.zeros(2)
torch.ones(3)
torch.linspace(1, 2, 3)
torch.arange(1, 2, 0.5)
```

9.2 `autograd.grad`

The whole point of `PyTorch` is that you can get the derivative of some calculated expression. To get that you indicate that your tensor is part of a graph by adding `requires_grad=True` when defining it, and you use `torch.autograd.grad` like this:

```
ps = torch.tensor([1.0, 2, 3], requires_grad=True)
qs = ps**2
print(ps)
print(qs)
dq_dp = torch.autograd.grad(qs, ps, torch.ones_like(ps))[0]
print(dq_dp)
```

```
tensor([1., 2., 3.], requires_grad=True)
tensor([1., 4., 9.], grad_fn=<PowBackward0>)
tensor([2., 4., 6.])
```

If you inspect the numbers you can see that PyTorch evaluated:

$$p = \begin{bmatrix}1, & 2, & 3\end{bmatrix}$$

$$q = p^2$$

$$\frac{dq}{dp} = 2p$$

Beware:

- The `ps`-tensor deliberately has a non-integer element in its definition as autodifferentiation requires floating-points. This could also be achieved by writing:

  ```
      ps = torch.tensor([1, 2, 3], requires_grad=True,
                        dtype=torch.float32)
  ```

- The `ps` and `qs` tensors are now part of graphs, and as such they cannot be used directly for e.g. plotting. See more in the next section.

- The `torch.autograd.grad` method can do a lot of things, and by providing the third argument, `torch.ones_like(ps)`, we force it to return:

  $$\begin{bmatrix}\frac{dq_0}{dp_0}, & \frac{dq_1}{dp_1}, & \ldots \frac{dq_N}{dp_N}\end{bmatrix}$$

  which is the desired $q(p)$ evaluated at $p = \begin{bmatrix}p_0, & p_1, & \ldots p_N\end{bmatrix}$ and does not contain elements such as $\frac{dq_i}{dp_j}, i \neq j$ that `torch.autograd.grad` can also calculate.

- The`[0]` after `torch.autograd.grad` is required since a tuple with the derivative inside is returned.

- `torch.autograd.grad` is a wrapper that does a number of things. You can actually build the derivative element-by-element in the following way:

  ```
  ps = torch.tensor([1.0, 2, 3])
  qs = []
  dq_dps = []
  for p in ps:
      p.requires_grad_(True)
      q = p**2     # this is the forward-part of the derivative
     evaluation
      q.backward()
      dq_dp = p.grad
      qs.append(q.detach())
      dq_dps.append(dq_dp.detach())
  print(ps)
  print(qs)
  print(dq_dps)
  ```

and get the same result:

```
tensor([1., 2., 3.])
[tensor(1.), tensor(4.), tensor(9.)]
[tensor(2.), tensor(4.), tensor(6.)]
```

9.3 with torch.no_grad()

As noted above, torch tensors that are part of a graph cannot be used directly for e.g. plotting with matplotlib. This can be remedied in various ways, e.g.:

```
fig, ax = plt.subplots()
with torch.no_grad():
    ax.plot(ps,qs)
```

or:

```
fig, ax = plt.subplots()
ax.plot(ps.detach(),qs.detach())
```

9.4 Higher derivatives

If you find the first derivative with `autograd.grad` with the intension of later on taking higher derivatives, you need to tell `autograd.grad` to build a graph while evaluating.

Without telling, the first derivative has no graph:

```
xs = torch.tensor([1.0, 2, 3], requires_grad=True)
fs = xs**3 / 3
dfdxs_no_graph = torch.autograd.grad(fs, xs,
                                     grad_outputs=torch.ones_like(xs))[0]
print(dfdxs_no_graph)
```

```
tensor([1., 4., 9.])
```

But adding `create_graph=True` to `autograd.grad` it has and the second derivative can be taken:

```
fs = xs**3 / 3 # must be calculated a 2nd time since the above
               # torch.autograd.grad already did a backward pass
dfdxs = torch.autograd.grad(fs, xs, grad_outputs=torch.ones_like(xs),
                            create_graph=True)[0]
d2fdx2s = torch.autograd.grad(dfdxs, xs,
                              grad_outputs=torch.ones_like(xs))[0]
print(dfdxs)
print(d2fdx2s)
```

```
tensor([1., 4., 9.], grad_fn=<MulBackward0>)
tensor([2., 4., 6.])
```

9.5 torch.nn.Module I

Whenever you want to use PyTorch for optimization, it is convenient to introduce a model using the `torch.nn.Module` class. An example implementing a method fitting a straight line would look like this:

```
class Line(torch.nn.Module):

    def __init__(self, w=1, b=0):
        super().__init__()
        self.b = torch.nn.Parameter(torch.tensor(float(w)))
        self.w = torch.nn.Parameter(torch.tensor(float(b)))

    def forward(self, x):
        y = self.w * x + self.b
        return y
```

The values for `w` and `b` are set during instantiation. The `torch.nn.Parameter()` statements take care of storing these variables with the `.requires_grad=True` setting so that autodifferentiation can be done on whatever the model (and outside code-pieces) calculate based on these parameters.

The model has a `forward`-method, which is the one that is used if the model is called. Here, the method receives some x-values and calculates what the linear function $f : f(x) = wx + b$ would give for them with the current values of w and b. A tensor with the results is returned.

As an example, we now let the straight line be defined by four data points without noise:

```
xs = torch.arange(1, 2, 0.25)
w_target = 4
b_target = -2
ys =  w_target * xs + b_target
xs, ys
```

```
(tensor([1.0000, 1.2500, 1.5000, 1.7500]), tensor([2., 3., 4., 5.]))
```

Instantiating the model, we can pass it our x-values and see what it predits with its initial parameters.

```
line = Line()
line(xs)
```

```
tensor([1., 1., 1., 1.], grad_fn=<AddBackward0>)
```

which matches that the model has $w = 1$ and $b = 0$ initially.

An optimizer is introduced:

```
optimizer = torch.optim.Adam(line.parameters(), lr=0.01)
```

and the `line.parameters()` lets our instance of the model `Line` inform the optimizer about what parameters may be optimized.

With the optimizer instantiated and informed about free parameters, a loop can be conducted in which:

- the optimizer prepares for a new calculation of the gradient of a loss function

- the model calculates the $f(x)$ for all x-values and a loss function is contructed:

$$\mathcal{L}oss = \sum_i |f(x_i) - y_i|^2$$

  where $\{y_i\}$ are the known function values corresponding to $\{x_i\}$.

- autodifferentiation is used to calculate the gradient of the loss with respect to the optimizable parameters (here with respect to `w` and `b` on the model):

$$\left(\frac{d}{dw}\mathcal{L}oss, \frac{d}{db}\mathcal{L}oss\right)$$

- the optimizer updates the optimizable parameters

It looks like this:

```
steps = 10000
for step in range(steps):
    optimizer.zero_grad()

    ys_model = line(xs)
    loss = torch.mean((ys_model - ys)**2)
    loss.backward()

    optimizer.step()

    if step % 1000 == 0:
        print(f'Step {step}, Loss: {loss.item()}')
```

```
Step 0, Loss: 7.5
Step 1000, Loss: 0.345175564289093
Step 2000, Loss: 0.06778551638126373
Step 3000, Loss: 0.0036803651601076126
Step 4000, Loss: 2.5285717129008844e-05
Step 5000, Loss: 5.567969196818012e-09
Step 6000, Loss: 5.286437954055145e-12
Step 7000, Loss: 9.805489753489383e-13
Step 8000, Loss: 1.1368683772161603e-13
Step 9000, Loss: 1.1368683772161603e-13
```

After the completion of the loop, the optimized parameters can be inspected directly via the attributes we know they are stored under:

```
line.w, line.b
```

```
(Parameter containing:
 tensor(4.0000, requires_grad=True),
 Parameter containing:
 tensor(-2.0000, requires_grad=True))
```

Or one can do it more elegantly in this way:

```
for name, parameter in line.named_parameters():
    print(name,':',parameter.data)
```

```
b : tensor(-2.0000)
w : tensor(4.0000)
```

9.6 torch.nn.Module II

Here is a 2nd example of a PyTorch model used for optimization. This time, the objective is to find the center of mass of a set of (equally heavy) particles in one dimension.

The model looks like this:

```
class CM(torch.nn.Module):

    def __init__(self, cm=1):
        super().__init__()
        self.cm = torch.nn.Parameter(torch.tensor(float(cm)))

    def forward(self, x):
        distances = x - self.cm
        return distances
```

To use the model, we make eight random x-positions and let them have a center of mass of 5, so that we know what to expect from the optimization:

```
xs = torch.randn(8)
xs -= torch.mean(xs)
xs *= 2
xs += 5
xs
```

```
tensor([5.0007, 3.2004, 8.3075, 6.3952, 4.2812, 2.4183, 5.7610, 4.6357])
```

The model can now be instantiated and its forward-method called:

```
cm = CM()
cm(xs)
```

```
tensor([4.0007, 2.2004, 7.3075, 5.3952, 3.2812, 1.4183, 4.7610, 3.6357],
       grad_fn=<SubBackward0>)
```

An optimizer is introduced and made aware of the adjustable parameters in our instance of the CM class:

```
optimizer = torch.optim.Adam(cm.parameters(), lr=0.01)
```

Now, the optimization loop can be written and executed:

```
steps = 2000
for step in range(steps):
    optimizer.zero_grad()

    ds = cm(xs)               # all distances with sign
    loss = torch.mean(ds**2)  # mean of squared distances
    loss.backward()

    optimizer.step()

    if step % 200 == 0:
        print(f'Step {step}, Loss: {loss.item()}')
```

```
Step 0, Loss: 19.002288818359375
Step 200, Loss: 7.962760925292969
```

```
Step 400, Loss: 4.069424152374268
Step 600, Loss: 3.14914608001709
Step 800, Loss: 3.014554500579834
Step 1000, Loss: 3.0028884410858154
Step 1200, Loss: 3.0023036003112793
Step 1400, Loss: 3.0022873878479004
Step 1600, Loss: 3.0022873878479004
Step 1800, Loss: 3.0022873878479004
```

The center of mass is found correctly, which either a direct inspection of the cm-objects attribute shows:

```
cm.cm
```

```
Parameter containing:
tensor(5.0000, requires_grad=True)
```

Or as a more general inspection of "all" optimizable parameters on the model reveals:

```
for name, parameter in cm.named_parameters():
    print(name,':',parameter.data)
```

```
cm : tensor(5.0000)
```

9.7 torch.nn.Module III

Here is a 3rd example, where the optimization is: given $f(x)$, find x_0 such that:

$$f(x_0) = 0$$

The model class is naturally called `FZero` and could look like:

```
import torch

class FZero(torch.nn.Module):

    def __init__(self, f, x0=0):
        super().__init__()
        self.f = f
        self.x0 = torch.nn.Parameter(torch.tensor(float(x0)))

    def forward(self):
        return self.f(self.x0)
```

The model has one trainable parameter defined by:

```
self.x0 = torch.nn.Parameter(torch.tensor(float(x0)))
```

The value for `x0` is 0 or whatever is provided during instantiation. The `torch.nn.Parameter()` takes care of storing this variable with the `.requires_grad=True` setting that was introduced above.

The model has a `forward`-method, which is the one that is used if the model is called. For the present model, it applies the function it was given during instantiation, stored as `self.f`, to `self.x0`. This means that calling the model, one gets to know $f(x_0)$.

```
# the function for which we seek the nodes
f = lambda x: (x-8)*(x+2)

# instantiation
fzero = FZero(f)

# what is f(x0) (according to the forward-method on fzero)?
fzero(f)
```

```
tensor(-16., grad_fn=<MulBackward0>)
```

Now to have the model change the value of `x0` so as to find a value where the function has a node (= is zero), an optimizer is introduced, e.g. the Adam optimizer:

```
optimizer = torch.optim.Adam(fzero.parameters(), lr=0.01)
```

where `fzero.parameters()` lets the model tell what parameters can be optimized, and where `lr=0.01` is the learning rate.

With the optimizer instantiated and attached, a loop can be conducted in which:

- the optimizer prepares for a new calculation of the derivative (gradient with more parameters) of the loss function
- the model calculates the $f(x_0)$ for the currect x_0 and the value is used for calculating a loss function, e.g.:
$$\mathcal{L}oss = |f(x_0)|^2$$
- autodifferentiation is used to calculate the derivative of the loss with respect to the optimizable parameters (here with respect to `x0`):
$$\frac{d}{dx_0}\mathcal{L}oss$$
- the optimizer updates the optimizable parameter(s)

It looks like this:

```
steps = 2000
for step in range(steps):
    optimizer.zero_grad()

    loss = fzero()**2
    loss.backward()

    optimizer.step()

    if step % 200 == 0:
        print(f'Step {step}, Loss: {loss.item()}')
```

and gives this output:

```
Step 0, Loss: 256.0
Step 200, Loss: 10.921854972839355
Step 400, Loss: 0.003035914385691285
Step 600, Loss: 3.139174031829839e-09
```

```
Step 800, Loss: 9.094945629950502e-11
Step 1000, Loss: 9.094945629950502e-11
Step 1200, Loss: 9.094945629950502e-11
Step 1400, Loss: 9.094945629950502e-11
Step 1600, Loss: 9.094945629950502e-11
Step 1800, Loss: 6.963317422670201e-11
```

Inspection of the optimized parameter on the model goes via its attribute `x0`, which can be accessed directly:

```
fzero.x0
```

```
Parameter containing:
tensor(-2.0000, requires_grad=True)
```

Or more neatly in this way:

```
for name, parameter in fzero.named_parameters():
    print(name,':',parameter.data)
```

```
x0 : tensor(-2.0000)
```

And we have indeed found a node of the function.

The other node can also be found:

```
fzero = FZero(f,x0=5.25)
optimizer = torch.optim.Adam(fzero.parameters(), lr=0.01)
steps = 2000
for step in range(steps):
    optimizer.zero_grad()

    loss = fzero()**2
    loss.backward()

    optimizer.step()

    if step % 200 == 0:
        print(f'Step {step}, Loss: {loss.item()}')

print('====')

for name, parameter in fzero.named_parameters():
    print(name,':',parameter.data)
```

```
Step 0, Loss: 397.50390625
Step 200, Loss: 58.134864807128906
Step 400, Loss: 0.2609196901321411
Step 600, Loss: 2.786054028547369e-05
Step 800, Loss: 2.751218364238639e-09
Step 1000, Loss: 2.751218364238639e-09
Step 1200, Loss: 2.751218364238639e-09
Step 1400, Loss: 2.751218364238639e-09
Step 1600, Loss: 2.751218364238639e-09
Step 1800, Loss: 1.8417255498448526e-09
====
x0 : tensor(8.0000)
```